
Vers l'automatisation de la gestion du cycle de vie des jumeaux numériques

Gwendal Beaumont^{1,2}, Antoine Beugnard¹, Salvador Martínez¹, Christelle Urtado², Sylvain Vauttier²

1. IMT Atlantique, Lab-STICC (UMR 6285)

Technopôle Brest-Iroise CS 83818

29238 Brest Cedex 3, France

{gwendal.beaumont,antoine.beugnard,salvador.martinez}@imt-atlantique.fr

2. IMT Mines Alès, SyCoIA

6 Av. de Clavières

30100 Alès, France

{christelle.urtado,sylvain.vauttier}@mines-ales.fr

RESUME. Cet article est une synthèse de l'article : Beaumont, G., Beugnard, A., Martínez, S., Urtado, C., Vauttier, S. (2026). Towards Automating the Life Cycle Management of Digital Twins. In: Bork, D., Lukyanenko, R., Sadiq, S., Bellatreche, L., Pastor, O. (eds) *Conceptual Modeling. ER 2025. Lecture Notes in Computer Science*, vol 16189. Springer, Cham. https://doi.org/10.1007/978-3-032-08623-5_22

MOTS-CLÉS : Jumeaux numériques, Ingénierie dirigée par les modèles, DevOps, GitOps

Le terme *jumeau numérique* (JN) fait référence à une entité virtuelle qui représente, copie et peut contrôler un système, un processus ou un objet donné, appelé système de référence (RS). La nature du RS, ainsi que les objectifs du JN peuvent être multiples. Les JN ont pour avantage de permettre, entre autres, la visualisation, la simulation ou la prédiction de l'état du RS. L'intérêt pour cette technologie ne cesse de croître. Des propositions récentes visent à standardiser le développement des JN (Aissat *et al.*, 2025), pourtant notre expérience montre qu'ils sont généralement créés de manière *ad hoc* et sans réelle prise en compte de leur cycle de vie (Beaumont *et al.*, 2024). Ces lacunes représentent des freins à la modularité, la réutilisation et la gestion de ces JN. Cet article propose l'établissement d'un cycle de vie opérationnel du JN ainsi qu'un processus de génération automatique d'artefacts GitOps permettant l'automatisation dudit cycle.

Dans un premier temps, à partir de notre expérience et d’une enquête sur la littérature, nous formalisons un ensemble d’opérations élémentaires caractérisant la phase opérationnelle du cycle de vie des JN. Cette base, conçue pour être extensible selon les besoins spécifiques, se compose des opérations suivantes : *start*, *configure*, *connect*, *disconnect*, *synchronize*, *clone*, *save*, *stop*, *archive*, *restore* et *delete*. Ces opérations intègrent des arguments permettant de configurer leurs points de variabilité (voir Figure 1). Par ailleurs, nous observons que l’exécution de ces opérations révèle une interdépendance temporelle. Par exemple, il paraît inconcevable d’arrêter l’instance d’un JN sans l’avoir démarré au préalable. Ainsi, la Figure 1 présente une mise en œuvre concrète d’enchaînement de ces opérations. Enfin, un exemple est proposé au travers du cas d’utilisation sur une usine d’apprentissage *Fischertechnik*.

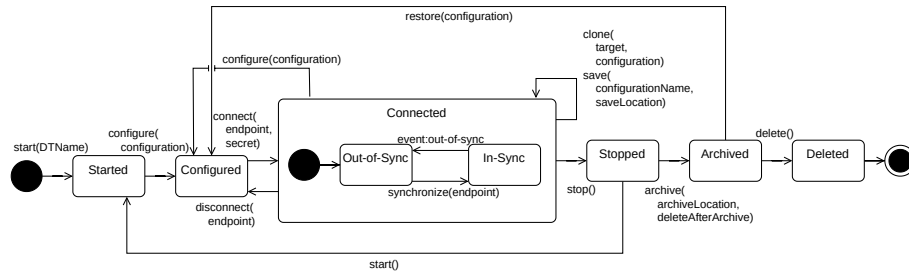


FIGURE 1. Cycle de vie opérationnel de JN sous la forme d’une machine à états

Dans un second temps, nous proposons l’automatisation de la gestion de ces opérations grâce à un processus intégrant des techniques d’ingénierie dirigée par les modèles appliquées à une approche *GitOps*. *GitOps* est l’application des principes *DevOps* à des outils basés sur le logiciel de gestion de version *Git*. Concrètement, après avoir défini un modèle conceptuel de l’approche *GitOps*, nous proposons un processus en quatre étapes permettant la génération d’artefacts pouvant être directement utilisés par les outils *GitOps*.

Étape 1 : Un modelleur de JN construit et produit des modèles du JN. Cette modélisation peut se réaliser via divers langages comme *Digital Twin Definition Language*¹ (DTDLD) ou *Eclipse Vorto*².

Étape 2 : En parallèle, un ingénieur *DevOps* / *GitOps* définit l’infrastructure de la plateforme sur laquelle sera installé le futur JN. Cela inclut le choix de plusieurs outils *GitOps*, dont un outil d’orchestration et un outil de conteneurisation. Ces derniers représentent les éléments cibles d’une plateforme *GitOps* et en assurent le fonctionnement. Tandis que les conteneurs contiennent les applications logicielles, l’orchestrateur coordonne et assure le fonctionnement des conteneurs. Dans cette étape sont produits des artefacts tels que la spécification de la plateforme et des outils.

1. <https://github.com/Azure/pendigitaltwins-dtdl>

2. <https://eclipse.dev/vorto/>

Étape 3 : Cette étape est une étape d’instanciation et requiert la complétude des étapes précédentes. Les fichiers produits précédemment sont analysés et décomposés afin d’instancier un modèle de JN et un modèle de plateforme, resp. *DT Model* et *Platform Model* dans la Figure 2. Ensuite, au travers de transformations de modèle à modèle, nous instancions un modèle de GitOps puis les modèles d’*orchestrateur* et de *conteneur*.

Étape 4 : Dans cette étape, les artefacts finals sont générés par des transformations de modèle à texte. Ils permettent de configurer, décrire et spécifier le comportement attendu de l’orchestrateur (stratégie de redondance, adaptation de la charge) et des conteneurs (outils installés, configuration des outils).

Un prototype d’implémentation basé sur l’usine Fischertechnik est en cours. Cet exemple est centré sur l’opération de connexion (*connect*) du cycle de vie et met en œuvre des outils comme *Message Queueing Telemetry Transport* (MQTT) pour la communication de données, *Kubernetes* en tant qu’orchestrateur et *Docker* en tant qu’outil de conteneurisation. Les transformations de modèle à texte sont réalisées avec *Acceleo* et celles modèle à modèle grâce à *ATL* et à l’environnement de modélisation d’Eclipse (*EMF*). Une vue globale de ce processus est proposée en Figure 2.

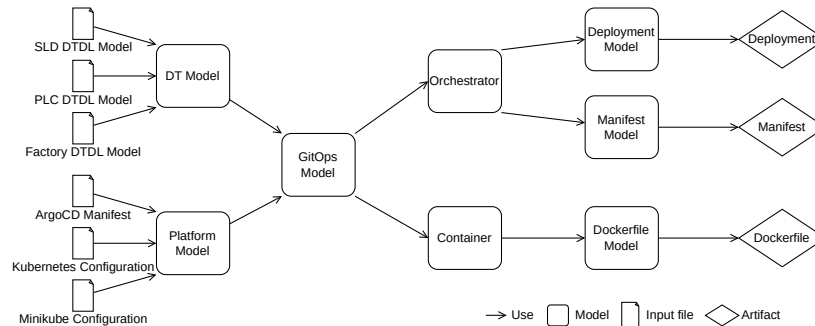


FIGURE 2. Vue globale du processus d’automatisation du cycle de vie

Ce papier propose un processus capable de prendre en compte le cycle de vie opérationnel d’un JN et d’en automatiser la gestion grâce à la génération d’artefacts logiciels pouvant être directement utilisés par des outils.

Bibliographie

- Aissat S., Beaulieu J., Poirier E., Motamedi A., Gascon-Samson J., Bordeleau F., « A devops framework for the systematic engineering and evolution of digital twins for built assets », *Software and Systems Modeling*, October, 2025.
- Beaumont G., Beugnard A., Martínez S., Urtado C., Vauttier S., « Towards Re-Engineering Digital Twins : Preliminary Experiments on Three Use Cases », *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, MO-DELS Companion '24*, Association for Computing Machinery, p. 453-458, October, 2024.